Reply to S. Danilov's comments on the mEVP communication study

N. Koldunov, 6 August 2026

On the intent of the divergence form. Understood, and it works as intended. Our two wide-halo variants differ only in what rides the window message, and on DARS at 32 GPUs the divergence form removes **28.8%** of the ice cost against **23.8%** for the classic form. Its advantage is ordered by how much mesh each GPU holds: it loses by 17 percentage points on CORE2 at 32 000 surface nodes per GPU, loses by 4 on fArc at 40 000, and gains 5 on DARS at 99 000. The halved, nodal message pays where messages are large, which is the regime it was aimed at.

Why our implementation exchanges more than yours. Not for bit-identity in the first instance, but because our ring is K cells deep and rings $2 \dots K$ do not exist in FESOM's data structures: on those nodes `a_ice`, `u_w`, `stress_atmice` and the right-hand sides are not last-bit different, they are absent. Your scheme never meets this because it stays inside the halo FESOM already maintains, which is also what restricts it to $K = 2$. At $K = 2$ the two approaches need the same data.

Point 2 — extending the computation over the $K - 1$ ring. You are right that this removes the error and makes the sub-domain boundaries invisible, and we have measured it. Averaging $|\Delta$ ice thickness$|$ on the sub-domain boundaries and in the interior over 60 days on fArc, the ratio is 0.87 for the configuration that recomputes the ring, against 0.85 for two identical runs; there is no imprint to detect. The configuration that does this is the one we call the wide halo.

The difficulty is that the same change removes most of the saving:

GPU, four meshes at their operating points	ice cost	boundary/interior ratio
delayed exchange, stale halo, no ring computation	−37 to −58%	1.70 (K=2), 2.52 (K=4)
ring recomputed (our wide halo)	−11 to −29%	0.87
two identical runs	—	0.85

The delayed exchange is about three times faster than the wide halo *because* it does not compute in the ring, and computing in the ring is what makes the boundaries invisible. The imprint and the saving have the same origin, so a scheme that extends the computation over the ring converges onto the wide halo and inherits its numbers. We have not found an intermediate that is both cheap and clean, and we would be glad to be shown one.

Point 1 — what sacrificing exactness would buy. Part of this we can bound and part we cannot.

The auxiliary fields we ship once per model step, which is the cost you object to, amount to 1.6–6.2% of the wide halo's messages and 3.6–9.2% of the data it moves, because they are sent once per step against $120/K$ window exchanges. Removing them altogether therefore gains at most about 0.4 percentage points of the model step. On this evidence the field shipping is not what holds the scheme back.

What we have not measured is the second half of exactness. Our ghost kernels reproduce the owner's summation order through a gather built for that purpose, and that runs every sub-cycle rather than once per step. It is the only place where a substantial gain could still be hiding, and we cannot bound it from the runs we have.

What we propose. Building the variant you describe — extend the computation over the ring, compute in the natural order, exchange velocities and stress divergence every K sub-cycles and recompute everything else — is a contained change to what already exists, and it answers your question with a number rather than an estimate. Since the ring computation is common to it and to our wide halo, we expect the difference between them to be the ordering machinery alone. We are happy to measure it, and to report the result whichever way it comes out.

All numbers above are from the runs described in `danilov_mevp_report.pdf`; the underlying output and job scripts can be shared.